

# 第八章 视觉SLAM

东北大学信息学院



- SLAM技术即同时定位与地图构建技术，一般可以分为激光SLAM技术和视觉SLAM技术。近年来，由于视觉传感器能够获得更加丰富的环境信息，体积小，成本低，基于视觉的SLAM技术逐渐受到关注。
- 视觉SLAM可以在没有任何环境先验信息的情况下，于运动过程中建立环境模型，同时估计自身位姿。以下列出了视觉SLAM的一些典型应用：扫地机器人、自动驾驶汽车、虚拟现实设备等。



- 在视觉SLAM“定位”与“建图”两个经典问题中，应用层面对于“定位”的需求是类似的，需要SLAM提供相机或搭载相机主体的空间位姿信息，而对于“建图”而言，则存在许多不同的需求，如导航、避障、三维重建以及人机交互等。



## 8.1 ROS操作系统

## 8.2 经典视觉SLAM框架

## 8.3 三维空间刚体运动描述

## 8.4 位姿估计

## 8.5 点云配准



# 8.1 ROS操作系统

## 8.1.1 ROS操作系统简介

## 8.1.2 ROS特点

## 8.1.3 ROS版本选择



## 8.1.1 ROS操作系统简介

- ROS是Robot Operating System的缩写，原本是斯坦福大学的一个机器人项目，后来由Willow Garage公司发展，目前由OSRF（Open Source Robotics Foundation, Inc）公司维护的开源项目。
- ROS虽然叫操作系统，但是真正底层的任务调度、编译、寻址等任务还是由Linux操作系统完成，也就是ROS是一个运行在Linux上的次级应用系统。
- ROS与传统的操作系统Windows、Linux以及Android是不同的，它更像是一个基于操作系统之上的一个软件包



## 8.1.2 ROS特点

- ROS主要有以下三大特点：

### (1) 点对点设计

- ROS采用点对点的设计，各个进程之间相互独立，这些进程存在于多个不同的主机并且在运行过程中通过端对端的拓扑结构进行联系，这样能够提高运行效率，并且便于对各个模块进行修改

### (2) 多语言支持

- 现如今编程语言越来越多，为了便于不同的用户使用，ROS支持多种编程语言。C++和Python已经在ROS中实现编译，是目前使用最多的ROS开发语言。同时C#和Java等语言也可支持



## 8.1.2 ROS特点

### (3) 免费开源

- ROS的所有源代码均是公开的，并且ROS还有专门的社区ROS WIKI (<http://wiki.ros.org>)。该网站不仅能够下载ROS安装包，并且还能搜索到有关ROS各种问题的解决方法以及功能包等，这为用户学习ROS提供了极大的方便。同时ROS以分布式的关系遵循BSD许可，允许各种商业和非商业的工程进行开发，这也促进了ROS的发展





## 8.1.3 ROS版本选择

- ROS1.0版本发布于2010年，经过多年的版本迭代，已发展多个版本



图 ROS各版本命名方式



## 8.1.3 ROS版本选择

- 不同的ROS版本中，对应的开发环境最稳定的为Ubuntu版本，下图列出了近几年使用的ROS版本以及对应的首选Ubuntu版本

Ubuntu和ROS版本对应关系及官方支持结束时间，加粗的为还在支持的版本，(Recommended)为官方建议使用版本。

| Ubuntu                                                                                        | ROS                                     | End of Life |
|-----------------------------------------------------------------------------------------------|-----------------------------------------|-------------|
| 14.04 LTS                                                                                     | indigo Igloo                            | April, 2019 |
| 16.04 LTS                                                                                     | Kinetic Kame                            | April, 2021 |
| 18.04 LTS                                                                                     | <b>Melodic Morenia</b>                  | May, 2023   |
| 20.04 LTS  | <b>Noetic Ninjemys</b><br>(Recommended) | May, 2025   |



## 8.2 经典视觉SLAM框架

### 8.2.1 SLAM分类

### 8.2.2 视觉SLAM框架



## 8.2.1 SLAM分类

- SLAM一般指搭载特定传感器的主体，在没有环境先验信息的情况下，在运动过程中能够建立环境模型，并能够同时估计自身的运动
- SLAM问题可以描述为：机器人在未知环境中从一个未知位置开始移动，在移动过程中根据位置估计和建图进行自身定位，并在此基础上建造增量式地图，从而实现机器人的自主定位和导航



## 8.2.1 SLAM分类

- 基于视觉的SLAM方法的分类：
- 1. 单目SLAM
- 2. 双目SLAM
- 3. RGB-D SLAM





# 单目SLAM

- 单目SLAM顾名思义使用的是单目相机获取图像信息和位置变换信息，其在日常生活中随处可见。但是单目相机在SLAM技术领域却有无法避免的缺点，主要在于单目相机拍摄得到的图像实际上是三维世界中的物体在二维平面上的投影，这导致从拍摄得到的平面图像中，无法得知图像中物体与相机之间的距离，即深度信息无法直接获取。因此，单目相机是通过移动相机并形成上下帧的视差从而获取深度信息的
- SLAM技术中前端的关键就在于即时定位，而单目SLAM显然并不能很好地实现这一点。另外，单目相机无法确定物体的真实尺度。如图所示，通过图片是无法判断手掌上人的尺度的



图 单目相机拍摄得到的图片



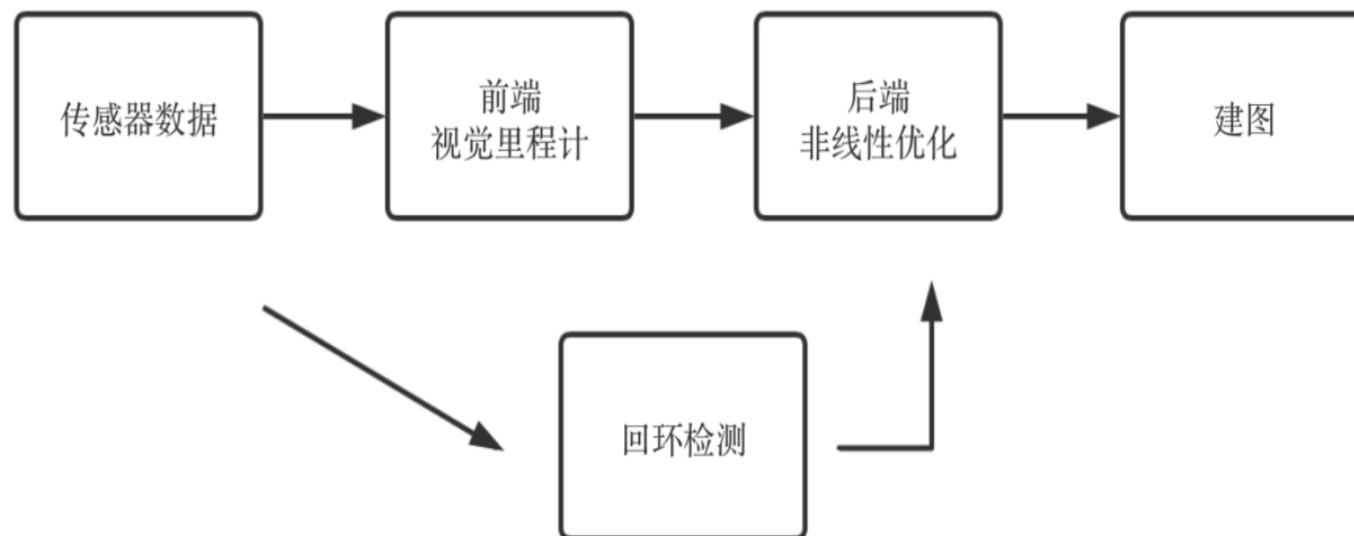
# 双目SLAM与RGBD-SLAM

- 双目SLAM技术顾名思义采用的为双目相机，相机的焦距、光圈中心点坐标位置、成像平面坐标和基线长度均已知，可以通过计算来获得每一幅图像的深度信息。但是双目相机存在的问题主要在于：双目相机在计算每个像素点的深度时计算量非常大；双目相机的测量范围与基线有关，基线距离越大，测得的距离就越远，这使得有些应用中，双目相机体积会很大，从而限制了应用
- RGBD-SLAM技术现如今应用较广。主要通过红外结构光或者Time-of-Flight向物体发射光线并接收返回的光图案来测量物体与相机之间的距离。RGB-D相机克服了单目相机无法直接获得深度信息以及双目相机计算量大的缺点，但是RGB-D相机也存在一些缺点，如易受光照干扰、测量范围较小等



## 8.2.2 视觉SLAM框架

- 视觉SLAM整体由四个模块组成：前端视觉里程计、后端非线性优化、回环检测以及建图。如下图所示：



图：视觉SLAM流程图





## 8.2.2 视觉SLAM框架

### 1. 传感器信息读取

- 在视觉SLAM中主要为相机图像信息的读取和预处理。如果在机器人中，还可能有码盘、惯性传感器等信息的读取和同步

### 2. 视觉里程计

- 视觉里程计是一个仅利用单个或多个相机的输入信息来估计智能体的运动过程。视觉里程计通过获取相邻两帧视图之间的 $\mathbf{R}_T$ 变换关系，将获取得到的多个 $\mathbf{R}_T$ 进行迭代优化
- 相比传统的里程计技术，视觉里程计更具优势。它只利用相机完成，无需场景和运动的先验信息；同时视觉里程计的成本较低，能够在水下和空中等GPS失效的环境中工作，其局部漂移率小于轮速传感器和低精度的惯性测量单元(Inertial Measurement Unit, IMU)；视觉里程计所获得的数据能够很方便地与其它基于视觉的算法融合，省去了传感器之间的标定



## 8.2.2 视觉SLAM框架

- 视觉里程计的实现过程如下：
  1. 输入图像序列
  2. 特征检测与特征匹配。对于获取的每一帧的新图像，首先要检测一些显著性强、重复性高的图像特征用于位姿估计。然后在相邻两帧图像之间进行特征匹配。特征匹配是视觉SLAM中极为关键的一步，其解决了SLAM中的数据关联问题，即确定当前看到的路标与之前看到的路标之间的对应关系，可为后期的位姿估计与优化等操作减轻大量负担
  3. 帧间位姿估计。首先排除特征点中不符合数学模型的异常数据，从而减少对位姿估计的影响。然后根据剩余下来的特征点计算相邻两帧之间相机的相对运动



## 8.2.2 视觉SLAM框架

### 3. 后端优化

- 后端接收不同时刻视觉里程计测量的相机位姿，以及回环检测的信息，对它们进行优化，得到全局一致的轨迹和地图。在实际环境中，即使是精确的传感器也会带有一定噪声，这些噪声会直接影响位姿估计的结果。后端优化主要解决的是从带有噪声的数据中估计整个系统的状态，以及这个状态估计的不确定性有多大。这里的状态包括机器人自身的轨迹，也包含地图



## 8.2.2 视觉SLAM框架

### 4. 回环检测

- 回环检测用于判断机器人是否曾经到达过先前的位置。如果检测到回环，它会把信息提供给后端进行处理。由于前端视觉里程计在进行位姿估计时，不可避免的会出现误差，这些误差会随着时间进行累计，从而造成机器人无法得知自己是否回到检测的起始点。通常采用判断图像间的相似性，来完成回环检测

### 5. 建图

- SLAM技术是一种底层技术，主要是为上层应用提供信息的。在增强现实设备中，可以利用SLAM技术将虚拟物体叠加在现实物体中。在视觉SLAM中，“建图”是服务于“定位”的。但是在应用层面来看，“建图”还有许多其它应用，比如定位、导航、避障、重建和交互



## 8.3 三维空间刚体运动描述

### 8.3.1 旋转矩阵和旋转向量

### 8.3.2 欧拉角



## 8.3.1 旋转矩阵和旋转向量

- 假如选取世界坐标系作为参考系，并选取机器人上的相机作为一个移动坐标系
- 相机坐标系会随着机器人的移动以及自身的旋转发生变化，因此可以通过相机坐标系相对于世界参考系的变化来描述机器人的位姿，即位置和姿态
- 位置变化可通过位置矢量来描述，姿态变化可通过旋转矩阵来描述



## 8.3.1 旋转矩阵和旋转向量

- 位置描述首先要选取坐标系，假如在三维空间中选取坐标系A，这样我们就能用 $3 \times 1$ 的位置矢量对坐标系A中的任何点进行定位
- 对于坐标系A下P点的位置，可以用一个矢量表示为：

$$\mathbf{P} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$





## 8.3.1 旋转矩阵和旋转向量

- 若是用一组单位正交基 $(\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3)$ 来表示该矢量则可以写为：

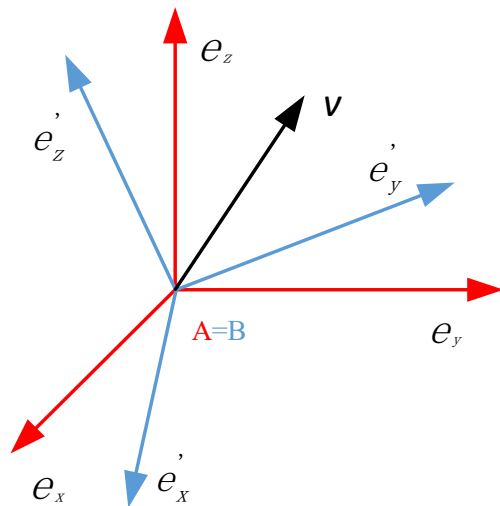
$$\mathbf{P} = [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3] \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} = p_1 \mathbf{e}_1 + p_2 \mathbf{e}_2 + p_3 \mathbf{e}_3$$

- 其中 $[p_1 \quad p_2 \quad p_3]^T$ 为向量 $\mathbf{P}$ 在坐标系A下的坐标。



## 8.3.1 旋转矩阵和旋转向量

- 姿态描述更加注重描述对象本身是否发生了变化。例如，相机的坐标系会随着相机自身的旋转发生变化。
- 如图所示，假设选取A坐标系与B坐标系分别作为相机旋转前后的坐标系，并且两个坐标系的单位正交基分别选取为 $[\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z]$ 和 $[\mathbf{e}'_x, \mathbf{e}'_y, \mathbf{e}'_z]$ 。



图：坐标系旋转



## 8.3.1 旋转矩阵和旋转向量

- 向量 $\boldsymbol{v}$ (该向量并没有随着坐标系的旋转而发生运动)在两坐标系下的坐标分别为 $[v_1 \ v_2 \ v_3]^T$ 和 $[v_1' \ v_2' \ v_3']^T$ , 则向量 $\boldsymbol{v}$ 可以表示为如下两种形式:

$$\boldsymbol{v} = [\boldsymbol{e}_x, \boldsymbol{e}_y, \boldsymbol{e}_z] \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \quad \boldsymbol{v} = [\boldsymbol{e}'_x, \boldsymbol{e}'_y, \boldsymbol{e}'_z] \begin{bmatrix} v_1' \\ v_2' \\ v_3' \end{bmatrix}$$

- 可得到如下等式:

$$[\boldsymbol{e}_x, \boldsymbol{e}_y, \boldsymbol{e}_z] \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = [\boldsymbol{e}'_x, \boldsymbol{e}'_y, \boldsymbol{e}'_z] \begin{bmatrix} v_1' \\ v_2' \\ v_3' \end{bmatrix}$$



## 8.3.1 旋转矩阵和旋转向量

- 由于 $[\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z]$ 为正交单位矩阵，则将等式两端分别左乘 $[\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z]^T$ ，可得到：

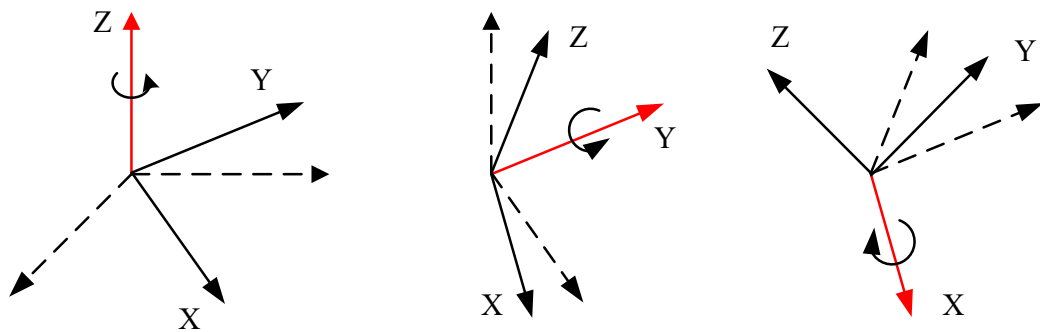
$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_x^T \\ \mathbf{e}_y^T \\ \mathbf{e}_z^T \end{bmatrix} [\mathbf{e}'_x, \mathbf{e}'_y, \mathbf{e}'_z] \begin{bmatrix} v'_1 \\ v'_2 \\ v'_3 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_x^T \mathbf{e}'_x & \mathbf{e}_x^T \mathbf{e}'_y & \mathbf{e}_x^T \mathbf{e}'_z \\ \mathbf{e}_y^T \mathbf{e}'_x & \mathbf{e}_y^T \mathbf{e}'_y & \mathbf{e}_y^T \mathbf{e}'_z \\ \mathbf{e}_z^T \mathbf{e}'_x & \mathbf{e}_z^T \mathbf{e}'_y & \mathbf{e}_z^T \mathbf{e}'_z \end{bmatrix} \begin{bmatrix} v'_1 \\ v'_2 \\ v'_3 \end{bmatrix} = \mathbf{R} \begin{bmatrix} v'_1 \\ v'_2 \\ v'_3 \end{bmatrix}$$

- 称式中的 $\mathbf{R}$ 为旋转矩阵



## 8.3.2 欧拉角

- 欧拉角使用三个分离的转角，把一个旋转分解成三次绕不同轴的旋转。可以先绕固定坐标系A的Z轴旋转，然后绕移动坐标系B的Y轴旋转，最后绕移动坐标系B的X轴旋转，我们称之为Z-Y-X欧拉角
- 如图所示给出了上述旋转过程以及旋转结果，图中虚线为旋转前的坐标轴，实线为旋转后的坐标轴（分别对应偏航角、俯仰角、滚转角）



图： Z-Y-X欧拉角



## 8.4 位姿估计

8.4.1 位姿估计的研究意义

8.4.2 位姿估计传统方法

8.4.3 基于深度学习的位姿估计方法



## 8.4.1 位姿估计的研究意义

- 位姿估计是移动机器人研究中的一个关键问题，对于目标跟踪、机器人导航以及三维重建等具有重要意义
- 精准的位姿估计是机器人实现自主工作的前提条件，位姿估计可分为对自身的位姿估计以及对外部目标的位姿估计。对外部目标的位姿估计又称为目标跟踪，良好的位姿估计能够实现对目标的精准跟踪



## 8.4.1 位姿估计的研究意义

- 当机器人进行位姿估计并能够建立较为精准的运动轨迹时，会使导航规划的路径变得更为合理与准确，同时后期的建图结果也会更加精确
- 位姿估计在三维重建中也具有非常重要的地位。例如在大场景三维重建中，会出现由位姿估计的累积误差而导致的相机漂移和重建模型质量低的问题，前期良好的位姿估计能够提高后期三维重建的质量





## 8.4.2 位姿估计传统方法

- 求解位姿估计的方法通常根据相机类型以及已知的匹配点信息进行选择
  - ① 当相机为单目时，已知两帧图像中的2D像素坐标，可采用对极几何解决位姿估计问题
  - ② 当已知3D点以及其在相机上的投影位置时，可采用PnP解决位姿估计问题
  - ③ 当相机为双目或者RGB-D时，可直接得知或者间接求解出2D点的深度信息，从而得到两组3D点，此时可采用ICP(Iterative Closest Point)解决位姿估计问题
- 除了上述的传统方法，还可通过深度学习方法解决位姿估计问题



## 8.4.3 位姿估计传统方法

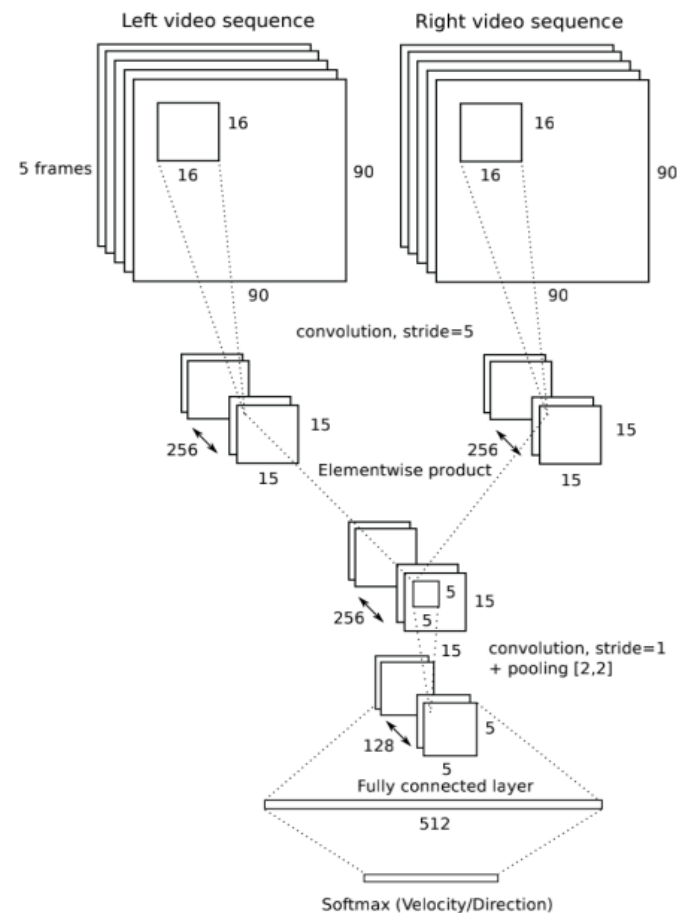
- 对于传统方法：
  - ① 2D-2D的对极几何方法需要较多点对（如八点法需要8个点对），且存在初始化、纯旋转和尺度的问题
  - ② 对于3D-2D的PnP方法，最少只需要3个点对就可以估计相机位姿。由于不需要使用对极约束，又可以在很少的匹配点中获得较好的位姿估计，是最重要的一种姿态估计方法
  - ③ 对于3D-3D的ICP方法，由于仅考虑两组3D点之间的变化，和相机模型并无关联。例如，在RGB-D SLAM中，可以用这种方法估计相机位姿



## 8.4.3 基于深度学习的位姿估计方法

### ① 监督学习方法

- 在监督学习方面，Konda 等人通过提取视觉运动和深度信息实现了端到端的基于深度学习的视觉里程计。在使用立体图像估计出深度信息之后，采用卷积神经网络预测相机速度和方向的改变
- 图中为两个具有相同框架的不同的神经网络，分别用来预测相机的速度和方向变化



图：预测相机速度和方向的卷积神经网络



## 8.4.4 基于深度学习的位姿估计方法

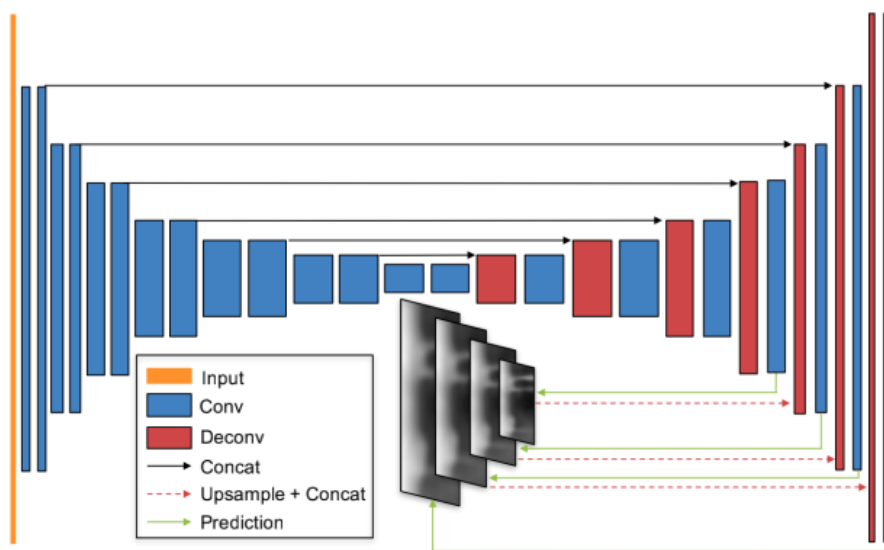
### ② 无监督学习方法

- 无监督学习在视觉里程计中应用较早，起初无监督学习在该领域的应用是提取稳定的特征点，通过这种特征点的匹配来求解相对位姿。随着深度学习的发展，目前逐步侧重在直接位姿估计方面
- Tinghui Zhou等通过无监督学习的方式可以同时估计出图像的深度、图像间的位姿状态以及图像中的动态物体。单帧图像通过Depth CNN网络获得深度图，相邻两帧图像通过 Pose CNN 生成相对的相机位姿，根据深度图与位姿将源图像投射到目标图像上，最后通过真实目标图像与投射产生目标图像的重建误差来训练网络

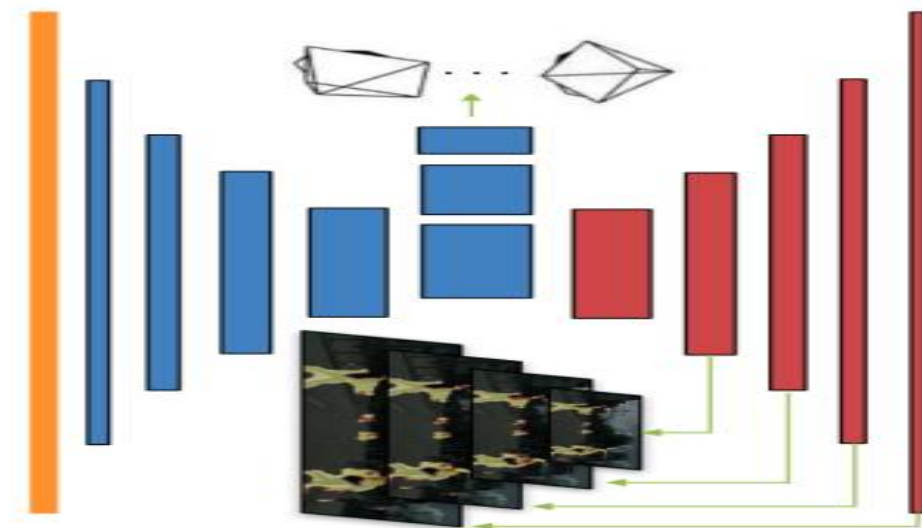


## 8.4.4 基于深度学习的位姿估计方法

- 如图 (a)所示，单视图深度预测网络架构 图 (b)为位姿/可解释性预测网络



(a) 单视图深度网络



(b) 位姿/可解释性深度网络

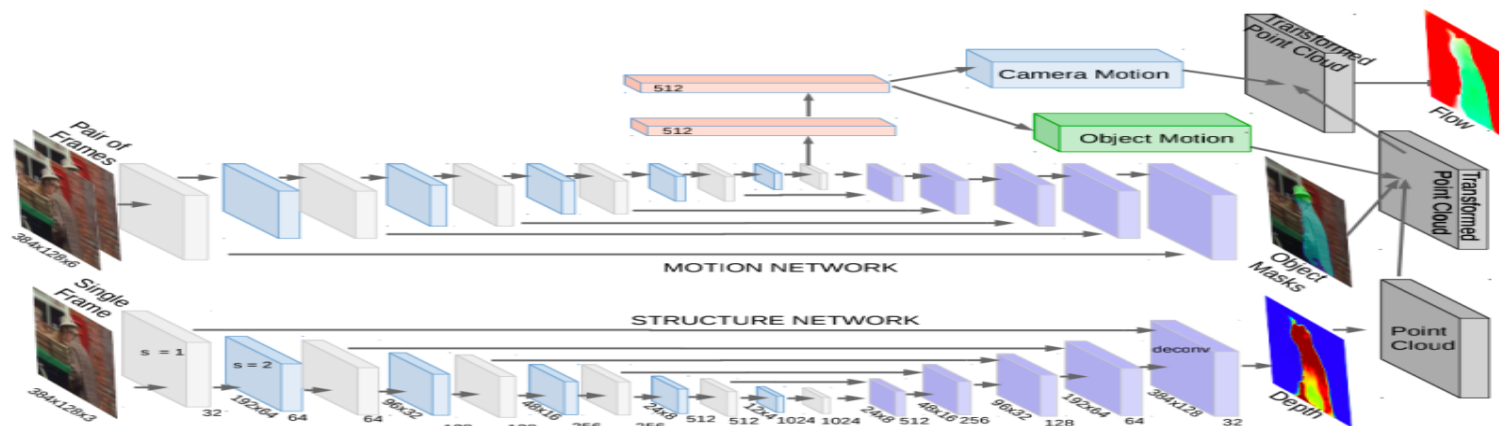
图：深度、位姿/可解释性预测网络



## 8.4.4 基于深度学习的位姿估计方法

### ③ 半监督学习方法

- 如图所示，SfM-Net首先对单帧图像进行卷积与反卷积操作得到图像的深度图并融合成深度点云。然后计算相邻两帧图像之间的变化位姿，识别出场景中的运动物体并进行图像分割，输出前景物体掩膜及其相对于背景的运动状态



图：SfM-Net结构图



## 8.5 点云配准

- 除了可以使用单目、双目相机进行稠密建图以外，由于深度相机可以直接获得点云信息、无需消耗大量的计算资源，而且点云数据精度高、快速匹配会使得最后的三维重建结果更加精准。因此，对于三维重建是一种更好的选择





## 8.5 点云配准

8.5.1 粗配准方案

8.5.2 精配准方案

8.5.3 结合法





## 8.5.1 粗配准方案

- 粗配准主要分为基于穷举搜索的配准算法与基于特征匹配的配准算法
- 基于穷举搜索的配准：RANSAC算法、4PCS算法等
- 基于特征匹配的配准：SAC-IA算法等



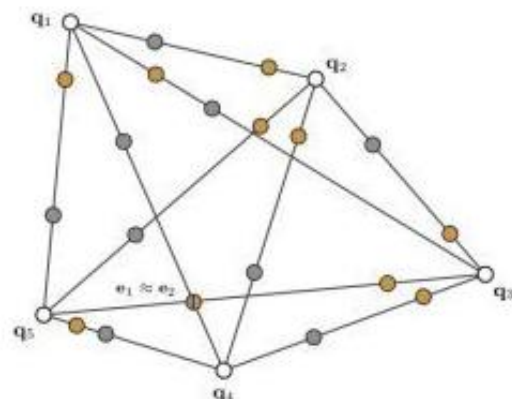
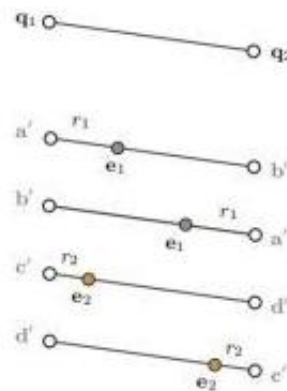
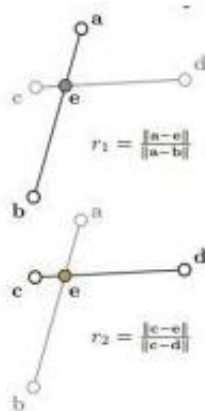
## 8.5.1 粗配准方案

- 4PCS配准算法使用的是RANSAC算法框架，通过构建与匹配全等四点对的方式来减少空间匹配运算，进而加速配准过程
- 输入点云P和Q构建共面四点集合，使用仿射不变性约束，在共面四点集合中匹配符合条件的对应点对，使用LCP（Largest Common Pointset）策略寻找配准后最大重叠度四点对，进而获得最优匹配结果，完成点云粗匹配



## 8.5.1 粗配准方案

- 4PCS算法构建与匹配点间空间拓扑关系过程如右上图所示：
- 首先在基准点云P中寻找满足要求的长基线（基线的确定与输入参数中overlap有很大关系，overlap越大，基线选择越长，长基线能够保证匹配的鲁棒性，且匹配数量较少）共面四点基，如图1用 $B=\{a,b,c,d\}$ 表示，然后提取共面四点基的拓扑信息，按式（1）计算四点基间的两个比例因子 $r_1$ 、 $r_2$ ，两比例因子在点云旋转和平移变化中具有仿射不变性。按式（2）计算 $q_1, q_2 \in Q$ 种可能存在的交点位置，进而计算所有Q中长基线点对交点坐标，比较交点坐标确定匹配集合， $e_i \approx e_j$ 表示寻找到对应的一致全等四点， $i, j$ 分别表示第 $i$ 个和第 $j$ 个Q中长基线点对。图1中 $B=\{a,b,c,d\}$ 的全等四点集为 $C=\{q_1, q_3, q_4, q_5\}$ 。寻找点云中所有P的共面四点集合记为 $E=\{B_1, B_2, \dots, B_m\}$ ， $m$ 为P中四点集总数，重复上述步骤可得到全等四点集合 $D=\{C_1, C_2, \dots, C_n\}$ ， $n$ 为全等四点集合总数



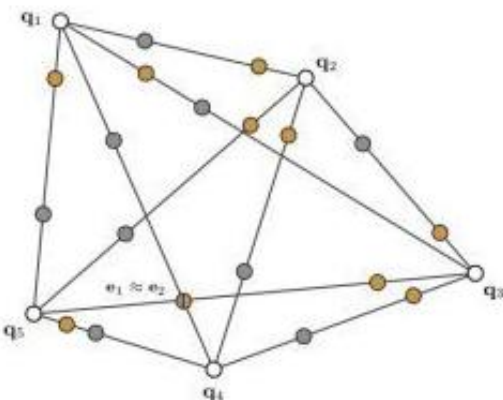
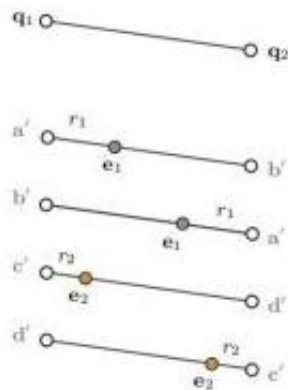
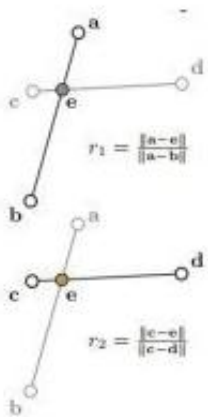
$$\begin{cases} r_1 = \|a - e\| / \|a - b\| \\ r_2 = \|c - e\| / \|c - d\| \end{cases} \quad (1)$$

$$\begin{cases} e_1 = q_1 + r_1(q_2 - q_1) \\ e_2 = q_1 + r_2(q_2 - q_1) \end{cases} \quad (2)$$



## 8.5.1 粗配准方案

- 4PCS算法构建与匹配点间空间拓扑关系过程如图1所示。首先在基准点云P中寻找满足要求的长基线（基线的确定与输入参数中overlap有很大关系，overlap越大，基线选择越长，长基线能够保证匹配的鲁棒性，且匹配数量较少）共面四点基，如图1用 $B=\{a,b,c,d\}$ 表示，然后提取共面四点基的拓扑信息，按式（1）计算四点基间的两个比例因子 $r_1$ 、 $r_2$ ，两比例因子在点云旋转和平移变化中具有仿射不变性。按式（2）计算 $q_1, q_2 \in Q$ 四种可能存在的交点位置，进而计算所有Q中长基线点对交点坐标，比较交点坐标确定匹配集合， $e_i \approx e_j$ 表示寻找到对应的一致全等四点， $i, j$ 分别表示第 $i$ 个和第 $j$ 个Q中长基线点对。图1中 $B=\{a,b,c,d\}$ 的全等四点为 $C=\{q_1, q_3, q_4, q_5\}$ 。寻找点云中所有P的共面四点集合记为 $E=\{B_1, B_2, \dots, B_m\}$ ， $m$ 为P中四点集总数，重复上述步骤可得到全等四点集合 $D=\{C_1, C_2, \dots, C_n\}$ ， $n$ 为全等四点集合总数



$$\begin{cases} r_1 = \|a - e\| / \|a - b\| \\ r_2 = \|c - e\| / \|c - d\| \end{cases} \quad (1)$$

$$\begin{cases} e_1 = q_1 + r_1(q_2 - q_1) \\ e_2 = q_1 + r_2(q_2 - q_1) \end{cases} \quad (2)$$



## 8.5.1 粗配准方案

- 在集合 $D=\{C1,C2,...,Cn\}$ 寻找最优全等四点对，4PCS使用LCP策略寻找最优全等四点匹配，即计算全等四点旋转和平移变化参数，将四点转化应用到全局点云转化，记录全局配准中包含最大的一致区域匹配记为最优匹配，至此完成4PCS算法局部粗配准工作



## 8.5.2 精配准方案

- ICP算法过程如下：
- 1. 设  $P = \{p_1, p_2, \dots, p_n\}$  与  $Q = \{q_1, q_2, \dots, q_n\}$  为初始待配准的两个点云数据集集合。
- 2. 对  $P$  中的每个点搜索其在  $Q$  中的最近点，常见的搜索方法有设定距离阈值加速查找与使用ANN加速查找。然后构建误差函数，如下式所示，其中  $q_i$  和  $p_i$  分别为对应最近点集中的第  $i$  对对应点：

$$E(\mathbf{R}, \mathbf{t}) = \frac{1}{n} \sum_{i=1}^n \|q_i - (\mathbf{R}p_i + \mathbf{t})\|^2$$



## 8.5.2 精配准方案

- 3. 通过计算 $\mathbf{R}$ 与 $\mathbf{t}$ 使得误差函数最小。通过去质心、奇异值分解方法计算旋转矩阵 $\mathbf{R}$ ，利用已求解的 $\mathbf{R}$ 计算 $\mathbf{t}$
- 4. 利用求解出的 $\mathbf{R}$ 、 $\mathbf{t}$ 对 $P$ 进行变换，得到新的点云数据集 $P' = \{p'_1, p'_2, \dots, p'_n\}$ ，并计算出 $P'$ 与 $Q$ 的平均距离

$$d = \frac{1}{n} \sum_{i=1}^n \|p'_i - q_i\|^2$$

- 5. 当 $d$ 小于某一设定阈值或者达到最大迭代次数时则停止计算，否则将新的点云数据集 $P'$ 代入第2步继续进行计算，直至满足迭代停止条件





## 8.5.3 结合法

- 在实际应用中，可以先对点云进行粗配准，为后期的精配准提供良好的初值。接着将粗配准后的点云再进行精配准，从而最终达到精准的配准效果。常见的方法有SAC-IA粗配准结合ICP精配准等





## 第八章小结

- ROS操作系统
- 经典视觉SLAM框架
- 三维空间刚体运动描述
- 位姿估计
- 点云配准
  - ICP算法流程